

PROJECT GUIDE

IMDB Movie Analysis Project

[Setup](#) • [Code Walkthrough](#) • [Dashboard](#) • [GitHub](#)

What you'll learn:

1. How to create a virtual environment and install dependencies
2. How to run the Jupyter notebook — line by line
3. How to build an interactive Streamlit dashboard
4. How to put the project on GitHub as a portfolio piece

***You will not need to type any code yourself.** All files are provided. This guide explains what each line does so you understand the project well enough to talk about it in an interview.*

Contents

#	Chapter	What it covers
1	Project Overview	What we're building and the folder layout
2	Setup — Virtual Environment	Create a venv, activate it, install dependencies
3	Notebook Walkthrough	The Jupyter analysis explained line by line
4	Streamlit Dashboard Walkthrough	The web app code explained line by line
5	Running the Project	What to run first, second, third
6	Putting it on GitHub	Make it a portfolio piece
7	Troubleshooting	Common errors and fixes

Chapter 1 — Project Overview

This is a complete data analytics project. You will analyse 1000 movies, find patterns, draw charts, and present everything in an interactive web dashboard. By the end, you'll have a real project to show on your GitHub profile.

1.1 What's in the project folder

```
imdb_project/
├──
│   ├── README.md           ← project description for GitHub
│   ├── requirements.txt    ← Python libraries needed
│   ├── .gitignore         ← files Git should ignore
│   ├──
│   ├── data/
│   │   ├── imdb_movies.csv ← 1000 movies dataset
│   │   ├──
│   │   ├── notebooks/
│   │   │   ├── imdb_analysis.ipynb ← Jupyter analysis notebook
│   │   │   ├──
│   │   │   ├── app/
│   │   │   │   ├── streamlit_app.py ← interactive web dashboard
│   │   │   │   ├──
│   │   │   │   ├── docs/
│   │   │   │   │   ├── setup_guide.pdf ← this document
```

1.2 The dataset

Our CSV file contains 1000 movies with these columns:

Column	What it contains	Example
title	Movie name	"The Dark Throne"
year	Release year	2017
genre	Movie genre	"Action"
director	Director's name	"A. Sharma"
runtime_minutes	Movie length	125
rating	Score out of 10	7.4
votes	Number of votes	85,000
budget_millions	Budget in million \$	45.0
revenue_millions	Revenue in million \$	180.5

1.3 What you'll build

Two deliverables:

- **Jupyter Notebook** — a step-by-step analysis with 10 charts. This is the 'study' version where you understand the data.
- **Streamlit Dashboard** — an interactive web app showing the same charts, with filters. This is the 'presentation' version.

WHY BOTH » Recruiters love projects with **both**. The notebook proves you can analyse data; the dashboard proves you can present results in a form non-technical people can use.

Chapter 2 — Setup: Virtual Environment

2.1 What is a virtual environment, and why?

A **virtual environment** (venv for short) is an isolated workspace for your project. Inside it, you can install Python libraries without affecting the rest of your computer. Each project gets its own venv.

Why do we need one?

- **Avoids conflicts.** Project A needs pandas 1.5, project B needs pandas 2.0. Without venvs, you can only have one installed at a time.
- **Keeps your computer clean.** When you delete the project, the venv goes with it. No leftover libraries cluttering your system.
- **Reproducible.** Anyone else who clones your project can recreate the exact same environment from `requirements.txt`.

IMPORTANT » Every professional Python project uses a virtual environment. It's not optional — it's a standard practice. Get used to it now.

2.2 Step-by-step setup

Step 1 — Open a terminal

First, open a terminal/command prompt:

- **Windows:** Press Win + R, type cmd, press Enter.
- **Mac:** Press Cmd + Space, type Terminal, press Enter.
- **Linux:** Press Ctrl + Alt + T.

Then navigate into the project folder using the `cd` command (change directory):

```
cd path/to/imdb_project

# For example, if it's on your Desktop:
# cd Desktop/imdb_project
```

Step 2 — Create the virtual environment

Type this command and press Enter:

```
python -m venv venv
```

What this does, word by word:

- `python` — runs Python
- `-m venv` — tells Python to run its built-in 'venv' module
- `venv` — the name of the folder to create (you can call it anything; 'venv' is the convention)

After this runs, you'll see a new folder called `venv/` inside your project. That's your virtual environment.

WATCH OUT » If you see `'python'` is not recognized, Python isn't installed or isn't in your system PATH. Install Python from python.org and tick the option **"Add Python to PATH"** during install. Or use `python3` instead of `python` on Mac/Linux.

Step 3 — Activate the virtual environment

Creating it isn't enough — you have to **activate** it so commands you run go into it. Activation is different on Windows vs Mac/Linux.

On Windows (Command Prompt)

```
venv\Scripts\activate
```

On Windows (PowerShell)

```
venv\Scripts\Activate.ps1
```

On Mac / Linux

```
source venv/bin/activate
```

After activation, your terminal prompt will change — it will show `(venv)` at the start. That's how you know it's active:

```
(venv) C:\Desktop\imdb_project>
  ↑
  this prefix tells you venv is active
```

TIP » Every time you open a fresh terminal to work on this project, you need to activate the venv again. Creating is one-time; activating is per session.

Step 4 — Install the dependencies

Now we install all the Python libraries the project needs. They're listed in `requirements.txt`. One command installs them all:

```
pip install -r requirements.txt
```

What this does:

- `pip` is Python's package installer
- `install` tells pip to install something
- `-r requirements.txt` tells pip to read the list of libraries from that file

This installs:

Library	What it's for
pandas	Working with tables of data
numpy	Numerical operations
matplotlib	Making charts

Library	What it's for
seaborn	Prettier charts on top of matplotlib
jupyter	Running the .ipynb notebook
streamlit	Building the web dashboard

Installation takes 1-3 minutes depending on your internet speed.

VERIFY » When done, type `pip list` to see everything installed. You should see all 6 libraries from the table above (plus their own sub-dependencies).

Step 5 — Deactivating the venv (when you're done)

To leave the venv at the end of a work session, just type:

deactivate

Your terminal prompt will lose the (venv) prefix. That's all there is to it.

Chapter 3 — Notebook Walkthrough (Line by Line)

This chapter walks through every section of `notebooks/imdb_analysis.ipynb`. Open the notebook side-by-side with this PDF and read both together. Run each cell with **Shift + Enter** while you read its explanation here.

3.1 Step 1 — Import the libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["font.size"] = 11
```

Line by line:

- `import pandas as pd` — bring in pandas; refer to it as **pd**
- `import numpy as np` — bring in numpy; refer to it as **np**
- `import matplotlib.pyplot as plt` — the plotting module from matplotlib
- `import seaborn as sns` — prettier charts
- `sns.set_theme(style="whitegrid")` — set a clean look with white background and grid
- `plt.rcParams["figure.figsize"] = (10, 6)` — default chart size for the whole notebook

3.2 Step 2 — Load the dataset

```
df = pd.read_csv("../data/imdb_movies.csv")
```

What's happening:

- `pd.read_csv(...)` — opens the CSV file and converts it to a DataFrame
- `"../data/imdb_movies.csv"` — the file path. `..` means "go up one folder" because the notebook is in `notebooks/` and the data is in `data/`
- `df = ...` — store the result in a variable called **df** (short for *data frame*)

3.3 Step 3 — The 4 first-look checks

On every new dataset, do these four checks in this exact order. They take 10 seconds and you know what you're working with.

Code	What it tells you
<code>df.shape</code>	Total rows and columns — (1000, 9)

Code	What it tells you
<code>df.head()</code>	First 5 rows so you can see what the data looks like
<code>df.info()</code>	Data types and missing value counts per column
<code>df.describe()</code>	Summary statistics for all numeric columns

TIP » `shape` has **no parentheses** because it's an *attribute* (a fact about the data). The others have parentheses because they're *methods* (they do work).

3.4 Step 4 — Clean missing values

```
median_runtime = df["runtime_minutes"].median()
df["runtime_minutes"] = df["runtime_minutes"].fillna(median_runtime)

median_budget = df["budget_millions"].median()
df["budget_millions"] = df["budget_millions"].fillna(median_budget)
```

What's happening:

- `df["runtime_minutes"].median()` — calculates the median of the runtime column
- `.fillna(value)` — replaces missing values with the given value
- We assign the result back (`df["runtime_minutes"] = ...`) to save the change

WATCH OUT » `fillna` returns a **new** Series with the values filled in. If you don't assign it back, your data stays unchanged. Always do `df["col"] = df["col"].fillna(...)`.

3.5 Step 5 — Top 10 highest-rated movies

```
popular = df[df["votes"] >= 10000]
top_10 = popular.sort_values("rating", ascending=False).head(10)
top_10[["title", "year", "genre", "rating", "votes"]]
```

Three operations chained together:

- `df[df["votes"] >= 10000]` — filter: keep only movies with at least 10,000 votes
- `.sort_values("rating", ascending=False)` — sort by rating, highest first
- `.head(10)` — keep the top 10 after sorting
- `top_10[...]` — display only the columns we care about

The bar chart code

```
plt.figure(figsize=(11, 6))
plt.barh(top_10["title"], top_10["rating"],
         color="steelblue", edgecolor="navy")
plt.title("Top 10 Highest-Rated Movies")
plt.xlabel("Rating (out of 10)")
plt.ylabel("Movie Title")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

Important lines:

- `plt.figure(figsize=(11, 6))` — set chart size (11 wide, 6 tall, inches)
- `plt.barh(labels, values)` — **horizontal** bar chart (good for long names)
- `plt.gca().invert_yaxis()` — flip so #1 appears at the top
- `plt.tight_layout()` — auto-adjust spacing so nothing overlaps
- `plt.show()` — actually display the chart

3.6 Steps 6–7 — Genre analysis

```
genre_counts = df["genre"].value_counts()
avg_rating_by_genre = df.groupby("genre")["rating"].mean().sort_values(ascending=False)
```

Key methods:

- `.value_counts()` — count how often each value appears, sorted from most to least
- `.groupby("genre")` — split the data into groups by genre
- `["rating"].mean()` — within each group, take the average rating

INFO » `groupby` is the most powerful Pandas tool. The pattern is *'group by X, then compute Y'*. Average marks per city, total sales per month, count of students per course — all use this pattern.

3.7 Step 8 — Movies per year (line chart)

```
movies_per_year = df["year"].value_counts().sort_index()

plt.plot(movies_per_year.index, movies_per_year.values,
         marker="o", linewidth=2)
```

Why `sort_index()` and not `sort_values()`?

- `sort_values()` would sort by COUNT (highest count first)
- `sort_index()` sorts by the INDEX (the year) — what we want for a time-series chart

3.8 Step 9 — Rating distribution (histogram)

```
plt.hist(df["rating"], bins=20, color="coral", edgecolor="black")

plt.axvline(df["rating"].mean(), color="red", linestyle="--",
            label=f"Mean = {df[\"rating\"].mean():.2f}")
plt.axvline(df["rating"].median(), color="blue", linestyle=":",
            label=f"Median = {df[\"rating\"].median():.2f}")
```

What's new:

- `plt.hist(values, bins=20)` — divide the rating range into 20 buckets
- `plt.axvline(x, ...)` — draws a vertical line at position x
- `f"...{value:.2f}"` — formats a number with 2 decimal places (called an *f-string*)

3.9 Step 11 — Budget vs revenue scatter plot

```
plt.scatter(df["budget_millions"], df["revenue_millions"],
            alpha=0.5, s=40, color="teal")
```

```
plt.plot([0, max_val], [0, max_val], "r--", label="Break-even")
```

Reading the chart:

- `alpha=0.5` — semi-transparent dots so overlaps are visible
- `s=40` — size of each dot
- The red dashed line is revenue = budget (break-even); dots above made profit, dots below lost money

3.10 Step 12 — Correlation heatmap

```
correlation_matrix = df[numeric_cols].corr()

sns.heatmap(correlation_matrix,
            annot=True, fmt=".2f",
            cmap="coolwarm",
            center=0, vmin=-1, vmax=1)
```

Settings explained:

- `annot=True` — write the correlation number in each cell
- `fmt=".2f"` — format numbers with 2 decimals
- `cmap="coolwarm"` — colour scheme: blue for negative, red for positive
- `vmin=-1, vmax=1` — colour scale range

3.11 Step 13 — Boxplot (rating spread per genre)

```
sns.boxplot(data=df, x="genre", y="rating",
            order=genre_order,
            palette="Set2")
```

Reading a boxplot:

- The **line in the box** = median rating for that genre
- The **box** = middle 50% of ratings (25th to 75th percentile)
- The **whiskers** = typical range of values
- **Dots outside whiskers** = outlier movies

3.12 Step 14 — Pie chart (revenue share)

```
plt.pie(revenue_by_genre.values,
        labels=revenue_by_genre.index,
        autopct="%1.1f%%",
        startangle=90)
```

Key parameters:

- `autopct="%1.1f%%"` — automatically write percentages inside each slice
- `startangle=90` — start the first slice at the top of the circle

TIP » Pie charts work best with 5-7 categories. With 10+ slices they get unreadable — switch to a horizontal bar chart instead.

Chapter 4 — Streamlit Walkthrough (Line by Line)

Streamlit turns a Python script into a web app. Open `app/streamlit_app.py` alongside this chapter. Every section of the file is explained below.

4.1 Section 1 — Imports

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

Standard imports. The convention for Streamlit is `st` — like `pd` for `pandas`.

4.2 Section 2 — Page configuration

```
st.set_page_config(
    page_title="IMDB Movie Dashboard",
    page_icon="",
    layout="wide",
    initial_sidebar_state="expanded",
)
```

Each setting:

- `page_title` — shown in the browser tab
- `page_icon` — emoji icon in the browser tab
- `layout="wide"` — use the full screen width (not centered)
- `initial_sidebar_state` — sidebar shown by default

WATCH OUT » `st.set_page_config` MUST be the first Streamlit command in the file. If anything else runs before it, you'll get an error.

4.3 Section 3 — Load the data (with caching)

```
@st.cache_data
def load_data():
    df = pd.read_csv("../data/imdb_movies.csv")
    df["runtime_minutes"] = df["runtime_minutes"].fillna(...)
    return df

df = load_data()
```

What's `@st.cache_data`?

It's a **decorator** — a special label that modifies how the function behaves. `@st.cache_data` tells Streamlit: *"remember the result of this function, don't recompute every time."*

Without caching, every time the user moves a slider, Streamlit would re-read the CSV file. With caching, it reads once and reuses the result — much faster.

4.4 Section 4 — Title and intro

```
st.title("IMDB Movie Data Dashboard")
st.markdown("An interactive dashboard exploring 1000 movies...")
st.divider()
```

Three useful Streamlit functions:

- `st.title(text)` — big heading at the top
- `st.markdown(text)` — write text with formatting (bold, italic, lists, links)
- `st.divider()` — horizontal line

Other heading levels:

- `st.header(text)` — section heading
- `st.subheader(text)` — sub-section heading

4.5 Section 5 — Sidebar with filters

```
st.sidebar.header("Filters")

year_range = st.sidebar.slider("Year range",
                                min_value=year_min, max_value=year_max,
                                value=(year_min, year_max))

selected_genres = st.sidebar.multiselect("Genres",
                                          options=all_genres, default=all_genres)

min_votes = st.sidebar.number_input("Minimum votes",
                                    min_value=0, value=0, step=1000)
```

Three widget types:

Widget	Used for	Returns
<code>st.slider</code>	Pick a value or range from a sliding bar	A number or tuple
<code>st.multiselect</code>	Pick multiple items from a dropdown	A list of selected items
<code>st.number_input</code>	Type a number	A number

The `.sidebar` **prefix** tells Streamlit to put the widget in the sidebar instead of the main page.

4.6 Section 6 — Apply the filters

```
filtered_df = df[
    (df["year"] >= year_range[0]) &
    (df["year"] <= year_range[1]) &
    (df["genre"].isin(selected_genres)) &
    (df["votes"] >= min_votes)
]
```

How this filter works:

- Each line is one condition wrapped in parentheses
- & means AND — all conditions must be true
- `.isin(list)` — checks if a value is in a list
- The result is a DataFrame with only matching rows

4.7 Section 7 — Metric cards (the scorecard)

```
col1, col2, col3, col4 = st.columns(4)

with col1:
    st.metric(
        label="Total Movies",
        value=f"{len(filtered_df):,}"
    )
```

Layout magic:

- `st.columns(4)` — splits the page into 4 equal columns side by side
- `with col1:` — everything indented inside goes in column 1
- `st.metric(label, value)` — special widget for big-number displays

TIP » `f"{value:,}"` formats a number with commas (e.g., 1000000 becomes "1,000,000"). The `:,` inside the braces is the format spec.

4.8 Sections 8–14 — The charts

Each chart follows the same pattern. Once you understand one, you understand all of them:

```
# 1. Create a matplotlib figure
fig, ax = plt.subplots(figsize=(10, 6))

# 2. Make the chart on `ax`
ax.bar(categories, values)
ax.set_xlabel("X label")
ax.set_ylabel("Y label")

# 3. Show it in the Streamlit app
st.pyplot(fig)

# 4. Free memory
plt.close(fig)
```

The key steps:

- `fig, ax = plt.subplots(...)` — create the chart canvas (fig) and the plot (ax)
- `ax.bar(...)` / `ax.scatter(...)` — draw the chart on the axes
- `st.pyplot(fig)` — display the matplotlib figure in the app
- `plt.close(fig)` — free memory after displaying

4.9 Section 15 — Expandable raw data

```
with st.expander("Show the raw data (click to expand)"):
    st.dataframe(filtered_df, use_container_width=True)
```

Two useful widgets:

- `st.expander(label)` — collapsible section users can click to expand
- `st.dataframe(df)` — interactive table users can sort and scroll

4.10 Streamlit cheat sheet

Function	What it does
<code>st.title(text)</code>	Page title
<code>st.header(text)</code>	Section heading
<code>st.subheader(text)</code>	Sub-section heading
<code>st.markdown(text)</code>	Text with formatting
<code>st.divider()</code>	Horizontal line
<code>st.columns(N)</code>	Split row into N columns
<code>st.metric(label, value)</code>	Big number display
<code>st.dataframe(df)</code>	Interactive table
<code>st.pyplot(fig)</code>	Display a matplotlib chart
<code>st.slider(...)</code>	Slider widget
<code>st.multiselect(...)</code>	Multi-pick dropdown
<code>st.expander(label)</code>	Collapsible section
<code>@st.cache_data</code>	Cache the result of a function

Chapter 5 — Running the Project

Now you know what each line does, here's the exact order to run things. Open a fresh terminal, then follow these steps.

Step 1 — Navigate to the project folder

```
cd path/to/imdb_project
```

Step 2 — Activate the virtual environment

Whatever OS you're on (Windows / Mac / Linux), use the right activate command:

```
# Windows:
venv\Scripts\activate

# Mac / Linux:
source venv/bin/activate
```

After this, your prompt shows (venv).

Step 3 — Run the Jupyter notebook

```
jupyter notebook notebooks/imdb_analysis.ipynb
```

A browser tab opens with the notebook. Press **Shift + Enter** to run each cell. Read the markdown, then run the code below it.

Run through every cell to see all 10 charts. When done, close the browser tab and stop the Jupyter server in your terminal with **Ctrl + C**.

Step 4 — Run the Streamlit dashboard

From the project root, type:

```
cd app
streamlit run streamlit_app.py
```

A browser tab opens at `http://localhost:8501` showing the dashboard.

TRY THIS » The dashboard has filters in the left sidebar. Play with them — every chart updates instantly when you change a filter. That's the magic of Streamlit.

When done, stop Streamlit with **Ctrl + C** in the terminal.

5.1 Summary: what runs in what order

When	What	Command
One time	Create virtual environment	<code>python -m venv venv</code>
Every session	Activate venv	Windows: <code>venv\Scripts\activate</code>

When	What	Command
One time	Install dependencies	<code>pip install -r requirements.txt</code>
Anytime	Run notebook	<code>jupyter notebook notebooks/imdb_analysis.ipynb</code>
Anytime	Run dashboard	<code>streamlit run app/streamlit_app.py</code>
End of session	Deactivate venv	<code>deactivate</code>

Chapter 6 — Putting It on GitHub

Once it works locally, put it on GitHub so you can show it on your resume, share with friends, and link it on LinkedIn.

6.1 Why GitHub?

- **Portfolio piece.** Recruiters look at your GitHub. A real project shows skills better than any certificate.
- **Backup.** Your code lives in the cloud — laptop crash doesn't kill it.
- **Collaboration.** Others can fork your project, suggest improvements, even contribute.

6.2 Step-by-step upload

1. Create a GitHub account

Go to <https://github.com> and sign up. It's free. Pick a professional username — recruiters will see it.

2. Install Git on your computer

Git is the tool that uploads your code to GitHub. Download from <https://git-scm.com/downloads> and install with defaults.

Verify the install:

```
git --version
```

3. Set up your Git identity (one time)

```
git config --global user.name "Your Name"  
git config --global user.email "your.email@example.com"
```

Use the same email as your GitHub account.

4. Create a new repository on GitHub

On the GitHub website:

- Click the + icon at the top-right
- Click **New repository**
- Name it `imdb-movie-analysis`
- Add a description: "Data analysis of 1000 movies with Streamlit dashboard"
- Choose **Public** (so anyone can see it)
- **Don't** tick "Initialize with README" (we already have one)
- Click **Create repository**

5. Upload your project

In your terminal, navigate to the project folder and run these commands one by one:

```
cd path/to/imdb_project

# Initialize Git in this folder
git init

# Stage all files for commit (the .gitignore excludes venv automatically)
git add .

# Take a snapshot with a description
git commit -m "Initial commit – complete IMDB analysis project"

# Set the main branch name
git branch -M main

# Connect to your GitHub repository
# Replace YOUR-USERNAME with your actual GitHub username
git remote add origin https://github.com/YOUR-USERNAME/imdb-movie-analysis.git

# Push your code to GitHub
git push -u origin main
```

It will ask for your GitHub username and password. For the password, GitHub now requires a **Personal Access Token (PAT)**, not your regular password. Create one at:

<https://github.com/settings/tokens>

6. Verify

Visit <https://github.com/YOUR-USERNAME/imdb-movie-analysis>. You should see all your files. The README appears below the file list automatically — that's the front page of your repository.

6.3 Bonus: Deploy the dashboard online (free)

Once your code is on GitHub, you can host your Streamlit dashboard online for free at streamlit.io. Anyone with the link can use it without installing anything.

- Sign up at <https://streamlit.io> with your GitHub account
- Click **New app**
- Pick your repository, branch, and file path (app/streamlit_app.py)
- Click Deploy. Wait 2-3 minutes.

You'll get a public URL like:

<https://imdb-movie-analysis.streamlit.app>

TIP » Add this URL to your README and your resume. Recruiters can click it and see your live work in 2 seconds — no installation needed. This is what makes Streamlit projects so powerful for portfolios.

Chapter 7 — Troubleshooting

Common errors and how to fix them. Save this page — you'll need it.

7.1 Setup errors

Error	What it means	Fix
python not recognized	Python isn't in your PATH	Reinstall Python from python.org, tick "Add to PATH"
pip not recognized	Same as above	Same fix — reinstall Python with PATH option
Permission denied during install	OS protecting system files	Make sure your venv is activated first
Cannot activate venv on PowerShell	Execution policy blocks scripts	Run: <code>Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser</code>

7.2 Jupyter / Pandas errors

Error	What it means	Fix
<code>FileNotFoundError: imdb_movies.csv</code>	Wrong file path	Check you're in the notebooks/ folder, and use <code>../data/imdb_movies.csv</code>
<code>KeyError: 'Marks'</code>	Wrong column name (case-sensitive)	Print <code>df.columns</code> to see exact names
<code>ModuleNotFoundError: pandas</code>	Library not installed in this venv	Activate venv, then <code>pip install -r requirements.txt</code>
Chart doesn't show in notebook	Missing display call	Add <code>plt.show()</code> at the end of the cell
<code>SettingWithCopyWarning</code>	Modified a slice of the DataFrame	Safe to ignore for this project

7.3 Streamlit errors

Error	What it means	Fix
streamlit not recognized	Streamlit not installed	<code>pip install streamlit</code>

Error	What it means	Fix
<code>set_page_config</code> can only be called once	Called more than once	Make sure it's the FIRST Streamlit command
Port 8501 already in use	Another Streamlit app is running	Stop it with Ctrl+C, or use <code>--server.port 8502</code>
Charts not updating with filters	Cache not invalidating	Press R (reload) in the browser, or restart the app
App is slow	Re-reading CSV every time	Make sure <code>@st.cache_data</code> is on <code>load_data()</code>

7.4 Git / GitHub errors

Error	What it means	Fix
git not recognized	Git not installed	Install from git-scm.com
fatal: not a git repository	Forgot to git init first	Run <code>git init</code> inside the project folder
Authentication failed	Wrong credentials	Use a Personal Access Token, not your password
Pushed too many files / venv folder	Forgot <code>.gitignore</code>	Make sure <code>.gitignore</code> exists with <code>venv/</code> in it BEFORE first commit
remote rejected – repository not found	Wrong remote URL	Double-check your username and repo name in the URL

You now have: a working virtual environment, a complete data analysis notebook, an interactive Streamlit dashboard, and a project on GitHub. That's a real portfolio piece — go add it to your resume!

Next steps:

- Apply this same workflow to a different Kaggle dataset
- Add more filters or charts to the Streamlit app
- Try predicting movie ratings with scikit-learn (next-level project)
- Deploy the dashboard online so you can share the live link